Полиморфизм (часть 2)

Александра Волосова,
к.т.н., доцент кафедры
ИУ5

Презентация разработана в рамках гранта «Грант на обучение студентов по образовательным программам высшего образования для топ-специалистов в сфере информационных технологий. Договор № 70-2025-000850 с АНО «Аналитический центр при Правительстве Российской Федерации»

В С++ полиморфизм достигается с помощью двух механизмов:

- Полиморфизм времени компиляции (перегрузка функций и операторов)
- **Полиморфизм времени выполнения** (виртуальные функции и динамическое связывание)

- Мономорфная функция — это функция способная применяться к **конкретному типу** данных.
- Полиморфная функция — это функция способная применяется к **различным типам** данных.

Открытая полиморфная функция

- это функция которая открыта для передачи в нее параметров любых типов, удовлетворяющих установленным ограничениям.

Для поддержки нового пользовательского типа достаточно чтобы он удовлетворял ограничениям функции и не требуется менять ее реализацию. Пользователь может расширять набор поддерживаемых типов без необходимости иметь доступ к реализации функции.

Закрытая полиморфная функция

- это функция которая закрыта для передачи в нее любых параметров заранее не поддержанных реализацией данной функции или набором функций.

Пример: **библиотека с полиморфной функцией** , принимающая один параметр:

- если для добавления поддержки нового типа параметра для этой функции нам не нужно менять саму функцию а достаточно чтобы новый параметр удовлетворял требования функции то это открытая полиморфная функция(open)
- если же мы не может поддержать новый тип без доступа к библиотеке и изменения ее реализации то это закрытая полиморфная функция(closed)

Closed type (перегрузка функций)

Перегрузка позволяет определять функции с одним и тем же именем, но разным набором параметров. Реализуется это за счет идентификации функции по совокупности имени и количеству и типу ее параметров.

C#

```
public virtual void Write(ulong value) { Write(value.ToString(FormatProvider)); }
public virtual void Write(float value) { Write(value.ToString(FormatProvider)); }
public virtual void Write(double value) { Write(value.ToString(FormatProvider)); }
public virtual void Write(decimal value) { Write(value.ToString(FormatProvider)); }
public virtual void Write(string? value) {
  if (value != null) { Write(value.ToCharArray()); }
}
public virtual void Write(object? value) {
  if (value != null) {
    if (value is IFormattable f){ Write(f.ToString(null, FormatProvider)); }
    else { Write(value.ToString()) };
  }
}
```

Closed type (перегрузка функций)

В Java отсутствия значений по умолчанию для параметров. Перегрузка помогает обойти этот недостаток:

// **Использование перегрузки** языками без параметров:

```
public static void Send(string body) {
  Send(body, "default");
}
public static void Send(string body, string to) {
  Send(body, to, 100);
}
public static void Send(string body, int timeOut) {
  Send(body, "default", timeOut);
}
public static void Send(string body, string to, int timeOut) {...}
```

// Если язык поддерживает параметры по , то **перегрузка не нужна**

```
public static void Send(string body, string to = "default", int timeOut = 100) {...}
```

Closed type (перегрузка операторов)

//Java, C#: операция над векторами

Vector3 complex = a.add(b).subtract(new Vector3(1, 1, 1)).multiply(0.5);

1. Объявляется переменная complex типа Vector3.
2. Выполняется сложение двух векторов a и b с помощью метода add.
3. Из результата вычитается вектор {1, 1, 1}, созданный с помощью конструктора new Vector3(1, 1, 1).
4. Результат умножается на 0.5 с помощью метода multiply.

//C#: операция над векторами

Vector3 complex = (a + b - new Vector3(1, 1, 1)) * 0.5;

Код демонстрирует **использование полиморфизма**, так как **методы add, subtract и multiply** могут быть определены для работы с **различными типами данных**, что позволяет выполнять арифметические операции над векторами

Полиморфизм на языке Kotlin - использования операторов и функций для работы с датами, списками и картами. Одни и те же операции могут быть применены к различным типам данных:

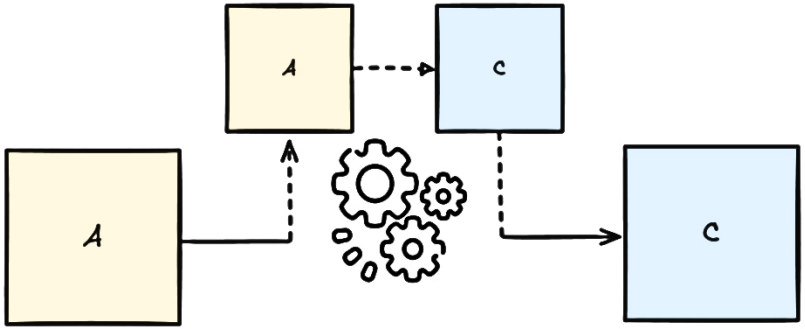
```
dateB > dateA                  // dateB.isAfter(dateA)
dateA == dateB                 // dateA.isEqual(dateB)
dateA < dateB                  // dateA.isBefore(dateB)
dateC in dateA..dateB          // dateA.rangeTo(dateB).contains(dateC)

listOf(1, 2, 3) + listOf(4, 5, 6) // listOf(1, 2, 3).plus(listOf(4, 5, 6))
listOf(1, 2, 3) + 4               // listOf(1, 2, 3).plus(4)
1 in listOf(1, 2, 3)              // listOf(1, 2, 3).contains(1)

map["One"]                        // map.get("One")
"One" in map                      // map.containsKey("One")
```

Используются операторы и функции для сравнения дат (dateB > dateA), проверки равенства (dateA == dateB), проверки вхождения элемента в диапазон (dateC in dateA..dateB), а также для работы со списками (listOf(1, 2, 3) + listOf(4, 5, 6)) и картами (map["One"])

Closed type (приведение типов)

Приведение осуществляется к типу большего размера

```
public static long Calculate(long n) {
  return n + 100 * 2;
}
byte b = 100;
short s = 100;
int i = 100;
long r1 = Calculate(b);
long r2 = Calculate(s);
long r3 = Calculate(i);
```



- Неявное приведение неявные приведения одних типов к другим.

```
public static Connection Connect(Address address) {...} // создание объекта перед вызовом функции
 var connection = Connect(new Address("192.168.1.1", 8453));
```

Если адрес приходит в виде строки: 192.168.1.1:8453, **то** нужно выполнить парсинг строки.

Так как строка содержит два разных типа данных: IP-адрес (строка) и Порт (число)

Если использовать перегрузку, то она нужна будет для каждой функции принимающей Address

Closed type (приведение типов)

Альтернативный вариант - использовать неявное приведение типа

```
public struct Address {
  public string Host { get; init; }
  public int Port { get; init; }
  public Address(string host, int port) {
    Host = host;
    Port = port;
  }
  public static implicit operator Address(string address) {
    Validator.Validate(address);
    var pair = address.Split(":");
    return new Address(pair[0], Convert.ToInt32(pair[1]));
  }
}
```

Ключевое слово implicit определяет оператор неявного преобразования. Тип string, переданный в функцию где требуется Address, будет неявно преобразован к нему с помощью данной функции

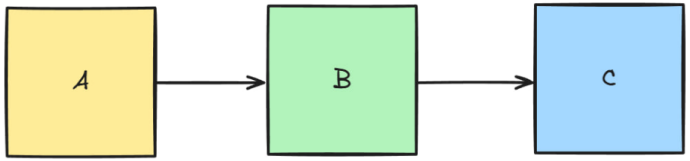
var connection1 = Connect(new Address("192.168.1.1", 8453));

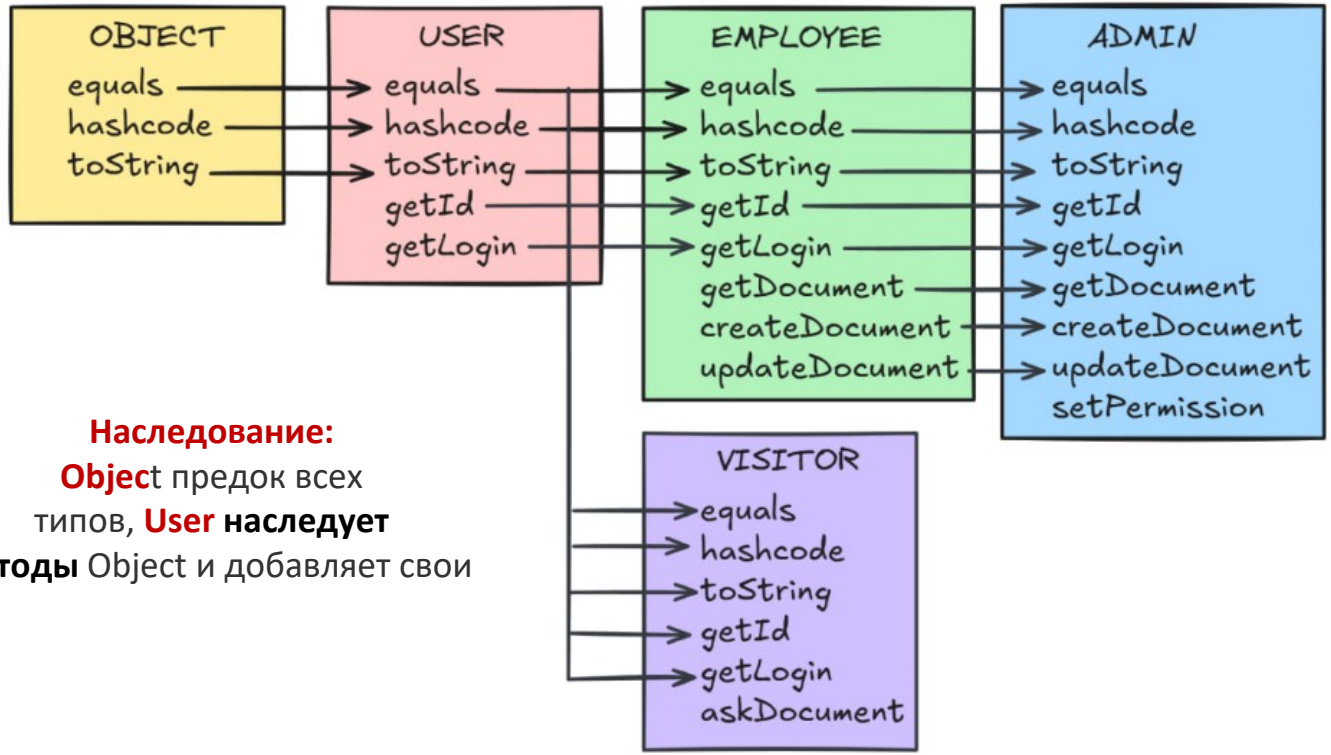
var connection2 = Connect("192.168.1.1:8453");

Любая функция требующая Address будет полиморфна по этому параметру, так как сможет принимать любой тип для которого Address реализовал **неявное преобразование**.

Opened type (полиморфизм подтипов)

Если тип B наследуется от типа A то публичные методы A переходят по наследству типу B и следовательно любой B можно подставить вместо любого A так как B это частный случай A



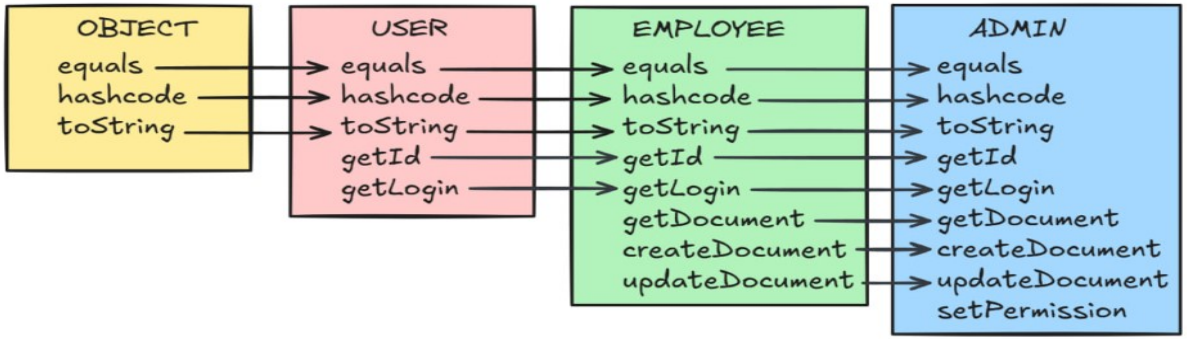


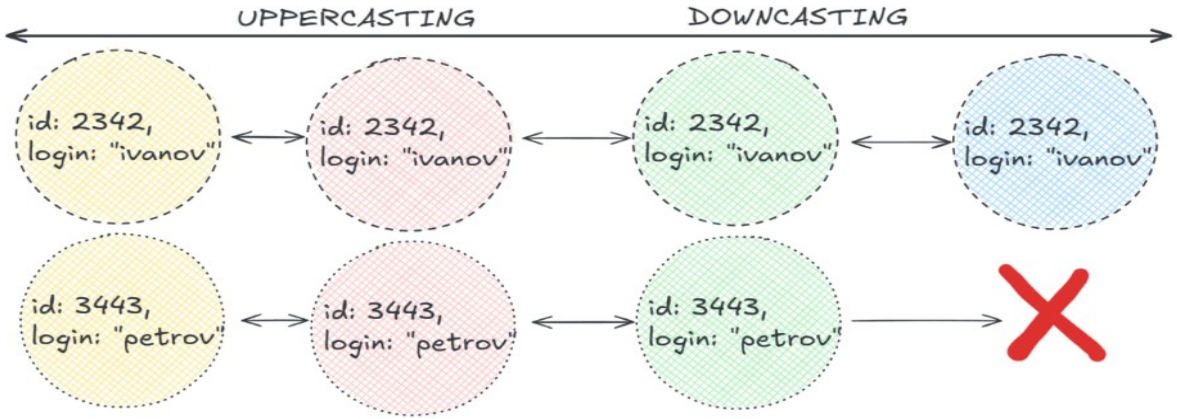
Наследование:
Object предок всех типов, **User наследует методы** Object и добавляет свои

Смеха роста офисного планктона

Opened type (полиморфизм подтипов)

В системах с **наследованием типов**, доступны операции приведения к родителю (upcasting) и приведение к дочернему типу (downcasting)





Объект с логином "ivanov" - администратор системы

- **Тип приведения upcasting** типа **Admin** до любого из его **родителей**:

чем выше будет тип родителя тем абстрактнее будет api объекта но тем меньшим количеством методов он сможет оперировать. Например если привести объект который имел тип Admin к его родителю - User то он потеряет как возможность назначать разрешения, доступную типу Admin так и возможность работать с документами доступную типу Employee.

- **Тип приведения downcasting** предполагает усиление возможностей типа за счет его уточнения:

объект "petrov", будучи изначально Employee, приведенный к родителю User, можно привести обратно к Employee или к любому из промежуточных типов

downcasting это небезопасная операция - если попытаться привести "petrov" к типу Admin, которому тот не соответствует, то произойдет ошибка

Opened type (полиморфизм подтипов)

Рассмотрим мономорфные функции, работающие с конкретными типами:

```
public static void PrintLogin(Visitor visitor) {
  Console.WriteLine(visitor.GetLogin()); }
public static void PrintLogin(Admin admin) {
  Console.WriteLine(admin.GetLogin()); }
```

Так как любой потомок умеет то что умеет родитель, любой объект можно как привести к родителю так и уточнить до изначального типа, то

Компилятор и среда выполнения может автоматически произвести upcasting любого дочернего объекта к необходимому родительскому типу и удовлетворить требование по типу функции.

Так как getLogin - это метод типа User, то при передаче любого потомка User аргумент будет приведен к User автоматически:

```
public static void PrintLogin(User user) {
  Console.WriteLine(user.GetLogin()); }
```

Функция уже полиморфна и работает универсально открыто с любыми типами – наследниками: Employee, Admin, Visitor

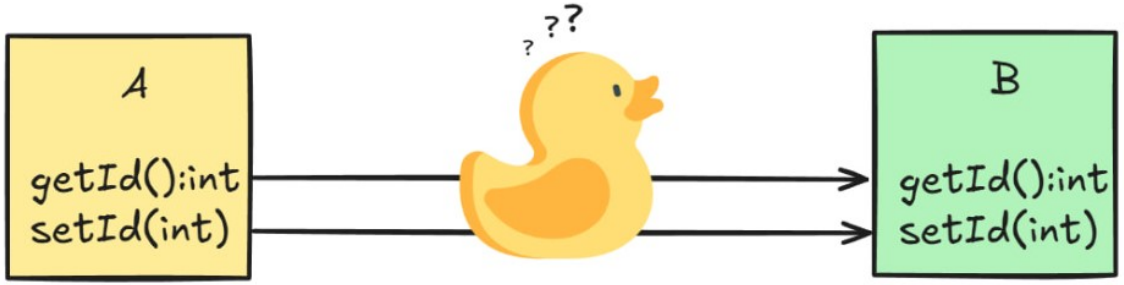
```
PrintLogin(new User());
PrintLogin(new Employee());
PrintLogin(new Admin());
PrintLogin(new Visitor());
```

Абстрагирование тип а приводит к потере функциональности

Уточнение типа приводит к увеличению функциональности и потере гибкости

Opene type (утиная типизация)

Если это выглядит как утка, плавает как утка и крякает как утка, то это, вероятно, и есть утка



- **Утиная типизация**: если тип содержит подходящие функции, этого достаточно чтобы им воспользоваться . Если функция Any требует тип B но в типе A имеются функции идентичные по наименованию, параметрам и возвращаемому значению, то в функцию Any можно передать тип A
- **Классическая типизация**: если Any принимает B то значит должен быть передан именно B и никак иначе.

C#

```
class ChessBoard {
  private readonly char[] _files = ['A', 'B', 'C', 'D', 'E', 'F', 'G',
'H'];
  private readonly int[] _ranks = [1, 2, 3, 4, 5, 6, 7, 8];
  public IEnumerator<string> GetEnumerator() {
    foreach (var rank in _ranks) {
      foreach (var file in _files) {
        yield return $"{file}{rank}"; } } } }
```

Для использования класса в цикле **foreach**, который работает только с коллекциями, необходимо для него реализовать интерфейс **Ienumerable**. Но класс **ChessBoard содержит метод GetEnumerator()** и этого достаточно для корректной работы, так как foreach реализует **утиную типизацию**

Opene type (утиная типизация)

Рассмотрим мономорфные функции, работающие с конкретными типами:

```
public static void PrintLogin(Visitor visitor) {
  Console.WriteLine(visitor.GetLogin()); }
public static void PrintLogin(Admin admin) {
  Console.WriteLine(admin.GetLogin()); }
```

Так как любой потомок умеет то что умеет родитель, любой объект можно как привести к родителю так и уточнить до изначального типа, то

Компилятор и среда выполнения может автоматически произвести upcasting любого дочернего объекта к необходимому родительскому типу и удовлетворить требование по типу функции.

Так как getLogin - это метод типа User, то при передаче любого потомка User аргумент будет приведен к User автоматически:

```
public static void PrintLogin(User user) {
  Console.WriteLine(user.GetLogin()); }
```

Функция уже полиморфна и работает универсально открыто с любыми типами – наследниками: Employee, Admin, Visitor

```
PrintLogin(new User());
PrintLogin(new Employee());
PrintLogin(new Admin());
PrintLogin(new Visitor());
```

Абстрагирование тип а приводит к потере функциональности

Уточнение типа приводит к увеличению функциональности и потере гибкости

Opene type (утиная типизация)

Go

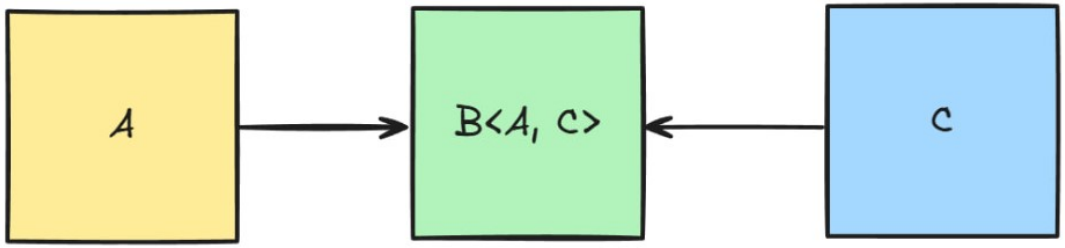
```
type Logger interface {
		Log(message string) }

type ConsoleLogger struct{}

func (c ConsoleLogger) Log(message string) {
		fmt.Println("[Console]", message) }

type FileLogger struct {
		File *os.File }

func (f FileLogger) Log(message string) {
		timestamp := time.Now().Format("2006-01-02
15:04:05")
		fmt.Fprintf(f.File, "[%s] %s\n", timestamp,
message)
}

func Process(logger Logger) {
		logger.Log("Программа запущена")
		logger.Log("Процесс выполняется")
		logger.Log("Готово!")
}
```

Утиная типизация:

в функцию Process можно передать любой объект,

у которого есть функция Log(message string)

Opene type (Параметрический полиморфизм) (дженерики)



обычная функция: входные параметры, позволяющие передавать в нее различные значения

параметрический полиморфизм: возможность передавать в функцию типы

C#

имя функции<блок параметров типов>(блок параметров значений)

```
public static IList<T> ToList<T>(T value) { return new List<T>() { value }; }
IList<int> list = ToList<int>(1);
```

Параметр типа T это заполняемый шаблон, передаваемый в функцию извне, условно преобразуя ее к виду :

```
public static IList<int> ToList<int>(int value) { return new List<int>() { value }; }
```

Один из типов был вынесен как шаблон что позволило передавать различные его вариации извне.

Параметризация - это абстрагирование от конкретных типов используемых функцией

позволяющая уточнять эти типы на этапе вызова

Open type (Параметрический полиморфизм) (дженерики)

функции которые принимают только параметры типов:

C#

```
public static IList<T> EmptyList<T>() {
  return new List<T>();
}

public static void PrintType<T>() {
  Console.WriteLine(typeof(T));
}
```

```
//вызовы
IList<int> intList = EmptyList<int>();
IList<string> strList = EmptyList<string>();

// System.Int32
PrintType<int>();
// System.String
PrintType<string>();
```

под капотом

```
public static IList<int> EmptyList<int>() {
  return new List<int>(); }
public static IList<string> EmptyList<string>()
{  return new List<string>(); }
public static void PrintType<int>() {
  Console.WriteLine(typeof(int)); }
public static void PrintType<string>() {
  Console.WriteLine(typeof(string)); }
```

```
IList<int> list = ToList<int>(1);

3 usages   new *
public static IList<T> ToList<T>(T value) {

    return new List<T>() { value };
}
```

Open type (Параметрический полиморфизм) (дженерики)

```
// Передан int логично что T == int и IList<T> == IList<int>
var list1 = ToList(1);

// Нельзя вывести тип так как нет входных параметров, нужно указывать явно
var emptyList = EmptyList<int>();

// Компилятор C# (Java) не сможет вывести  параметр типа из возвращаемого значения
// IList<int> emptyList2 = EmptyList();
```

TypeScript, Koltin, Rust

```
// TS
const emptyList1 = emptyList<number>();
const emptyList2: number[] = emptyList();
```

```
// Kotlin
val emptyList1 = emptyList<Int>()
val emptyList2: List<Int> = emptyList()
```

```
// Rust
let emptyList1 = emptyList::<i32>();
let emptyList2: Vec<i32> = emptyList();
```

Open type (Параметрический полиморфизм) (дженерики)

// Rust

```
let set = vec![1, 2, 3]
    .iter()
    .map(|n| n + 1)
    .collect::<HashSet<i32>>();
```

// Java

```
var set = List.of(1, 2, 3)
        .stream()
        .map(n -> n + 1)
        .collect(Collectors.toSet());
```

В **Java** нужно явно вызывать метод Collectors.toSet() чтобы указать в какую структуру данных преобразовать итератор. Благодаря системе типов в **Rust** достаточно прописать тип и реализация будет обеспечена

Полиморфизм: определение и виды

Полиморфизм (многоформенность):

- Возможность объектов разных типов отвечать на одинаковые сообщения

Виды полиморфизма в C++:

1. **Компиляционный** (статический):
   - Перегрузка функций
   - Шаблоны
2. **Выполняемый** (динамический):
   - Виртуальные функции
   - Наследование

Принципы полиморфизма в C++

Ключевые принципы:

- Принцип подстановки Лисков (LSP): объекты производного класса могут заменять объекты базового
- Позднее связывание: решение о том, какую функцию вызывать, принимается во время выполнения
- Указатели/ссылки на базовый класс: полиморфизм работает только через них

Пример полиморфного поведения

```
class Shape {
public:
    virtual void draw() const {
        std::cout << "Drawing a shape" << std::endl;
    }
    virtual ~Shape() {}
};

class Circle : public Shape {
public:
    void draw() const override {
        std::cout << "Drawing a circle" << std::endl;
    }
};

class Square : public Shape {
public:
    void draw() const override {
        std::cout << "Drawing a square" << std::endl;
    }
};
// Полиморфное использование
void drawAll(const std::vector<Shape*>& shapes) {
    for (auto shape : shapes) {
        shape->draw();  // Вызовется правильная версия draw()
    }
}
```

```
// Создание объектов
std::vector<Shape*> shapes;
shapes.push_back(new Circle());
shapes.push_back(new Square());
shapes.push_back(new Circle());
// Выведет: circle, square, circle
drawAll(shapes);
```

Виртуальные таблицы (vtable)

- Каждый полиморфный класс имеет свою vtable
- vtable содержит указатели на виртуальные функции
- Каждый объект содержит скрытый указатель (vptr) на vtable
- vptr устанавливается конструктором
- Вызов виртуальной функции:

1. Получить vptr из объекта
2. Найти нужную функцию в vtable
3. Вызвать функцию через указатель

Структура vtable (пример)

```
class Base {
public:
    virtual void f() { }
    virtual void g() { }
    virtual void h() { }
    int x;
};
```

```
// Память объекта Base:
// [vptr] → Base_vtable
// [x]    → значение x
// Base_vtable: [&Base::f, &Base::g, &Base::h]
```

```
class Derived : public Base {
public:
// Переопределяет Base::f
    void f() override { }
// Переопределяет Base::g
    void g() override { }
// Новая виртуальная функция
    virtual void i() { }
    int y;
};
```

```
// Память объекта Derived:
// [vptr] → Derived_vtable
// [x]    → значение x наследуется
// [y]    → значение y
// Derived_vtable: [&Derived::f, &Derived::g,
&Base::h, &Derived::i]

// vptr устанавливается конструктором:
// Base::Base() { vptr = &Base_vtable; }
// Derived::Derived() {
//   Base::Base();       // vptr = &Base_vtable
//   vptr = &Derived_vtable;
// }
```

Виртуальные функции и override (C++11)

virtual:

- Указывает, что функция виртуальная
- Может использоваться только в объявлениях
- Наследуется производными классами

override (C++11):

- Явно указывает, что функция переопределяет виртуальную функцию базового класса
- Помогает избежать ошибок: если нет совпадения, компилятор выдаст ошибку

final (C++11):

- Запрещает дальнейшее переопределение функции
- Может применяться к функциям или классам

Пример: override и final

```
class Base {
public:
    virtual void f(int) { }
    virtual void g() const { }
    virtual void h() { }
    void nonVirtual() { }  // Не виртуальная
};

class Derived : public Base {
public:
    // Правильно: переопределяет Base::f
    void f(int) override { }

    // Ошибка компиляции: сигнатура не совпадает
    // void f(double) override { }  // ERROR

    // Правильно
    void g() const override { }

    // final: запрещает дальнейшее переопределение
    void h() final { }

    // Ошибка: попытка override невиртуальной функции
    // void nonVirtual() override { }  // ERROR
};
```

```
class FurtherDerived : public Derived {
public:
    // Можно переопределить f и g
    void f(int) override { }
    void g() const override { }

// Нельзя переопределить h –
// она final в Derived
    // void h() override { }  // ERROR
};

// final класс: нельзя наследовать
class NoDerived final : public Base {
    // ...
};
// ERROR
class TryDerived : public NoDerived { };
```

Виртуальные деструкторы

Когда нужен виртуальный деструктор:

- Если класс предназначен для использования полиморфно (через указатель/ссылку на базовый класс)
- Если класс имеет хотя бы одну виртуальную функцию
- Страуструп: «Если класс имеет виртуальную функцию, у него должен быть виртуальный деструктор»
- Липпман: «Без виртуального деструктора удаление через указатель на базовый класс приводит к неопределенному поведению»

Проблема без виртуального деструктора:

- Удаление объекта производного класса через указатель на базовый
- Вызывается только деструктор базового класса
- Ресурсы производного класса не освобождаются

Пример виртуального деструктора

```
// Без виртуального деструктора
class BaseBad {
public:
    ~BaseBad() {
std::cout << "BaseBad destroyed" << std::endl; }
};

class DerivedBad : public BaseBad {
public:
    DerivedBad() { data = new int[100]; }
    ~DerivedBad() {
        delete[] data;
        std::cout
        << "DerivedBad destroyed" << std::endl;
    }
private:
    int* data;
};

// С виртуальным деструктором (ПРАВИЛЬНО)
class BaseGood {
public:
    virtual ~BaseGood() {
std::cout << "BaseGood destroyed" << std::endl; }
};
```

```
class DerivedGood : public BaseGood {
public:
    DerivedGood() { data = new int[100]; }
    ~DerivedGood() override {
        delete[] data;
        std::cout
        << "DerivedGood destroyed" << std::endl;
    }
private:
    int* data;
};

// Использование
BaseBad* bad = new DerivedBad();
// Только BaseBad::~BaseBad(), утечка памяти
delete bad;

BaseGood* good = new DerivedGood();
// Сначала DerivedGood::~DerivedGood(),
//затем BaseGood::~BaseGood()

delete good;
```

Абстрактные классы

Абстрактный класс:

- Класс, содержащий хотя бы одну чисто виртуальную функцию
- Не может быть инстанциирован (создан объект)
- Используется как интерфейс для производных классов
- Страуструп: «Абстрактные классы задают интерфейсы»
- Липпман: «Абстрактные классы реализуют концепцию интерфейса в C++»

Чисто виртуальная функция:

- Функция, объявленная с = 0
- Не имеет реализации в абстрактном классе
- Должна быть переопределена в конкретных производных классах
- Может иметь реализацию (используется редко)

Пример абстрактного класса

```
// Абстрактный класс (интерфейс)
class Shape {
public:
    // Чисто виртуальные функции
    virtual double area() const = 0;
    virtual void draw() const = 0;
    virtual std::string name() const = 0;

    // Виртуальный деструктор (обязательно!)
    virtual ~Shape() {}

    // Обычная виртуальная функция с реализацией
    virtual void printInfo() const {
        std::cout << name()
<< ", area = " << area() << std::endl;
    }

    // Статическая функция (не виртуальная)
    static int getShapeCount() { return count; }

protected:
    static int count;  // Счетчик созданных фигур
};
```

```
// Конкретный класс
class Circle : public Shape {
public:
    Circle(double r) : radius(r) {
    ++count; }
    double area() const override {
        return 3.14159 * radius * radius;
    }
    void draw() const override {
        std::cout
<< "Drawing circle with radius "
<< radius << std::endl;
    }
    std::string name() const override {
        return "Circle";
    }

    ~Circle() override { --count; }
private:
    double radius;
};
// Ошибка: абстрактный класс
Shape shape
// Верно: конкретный класс
Circle circle(5.0);
```

Порядок создания полиморфных объектов

Порядок при создании объекта:

1. Выделяется память для всего объекта
2. Инициализируются vptr базового класса
3. Вызывается конструктор базового класса
4. Инициализируются члены базового класса
5. Тело конструктора базового класса
6. Инициализируются vptr производного класса
7. Вызывается конструктор производного класса
8. Инициализируются члены производного класса
9. Тело конструктора производного класса

Виртуальные функции в конструкторах

- В конструкторе базового класса виртуальные функции вызываются как функции базового класса
- vptr еще не установлен на таблицу производного класса
- Страуструп: «Не вызывайте виртуальные функции в конструкторах»

Пример порядка создания полиморфных объектов

```
class Base {
public:
    Base() {
        std::cout << "Base constructor" << std::endl;
        f();  // Вызовет Base::f(), а не Derived::f()!
    }
    virtual void f() {
std::cout << "Base::f()" << std::endl; }
    virtual ~Base() {
std::cout << "Base destructor" << std::endl; }
};

class Derived : public Base {
public:
    Derived() {
        std::cout << "Derived constructor" << std::endl;
        f();  // Вызовет Derived::f()
    }
    void f() override {
        std::cout << "Derived::f()" << std::endl; }
    ~Derived() {
        std::cout << "Derived destructor" << std::endl; }
};
```

```
// Создание объекта
Derived d;
// Вывод:
// Base constructor
// Base::f()
// Derived constructor
// Derived::f()

// Уничтожение объекта (обратный
порядок):
// Derived destructor
// Base destructor

// Виртуальные вызовы в деструкторах:
~Base() {
    f();  // Вызовет Base::f() //даже
если объект Derived!
// vptr уже установлен на Base_vtable
}
```

Свойства виртуальных функций

Свойства и ограничения:

- Виртуальность наследуется
- Можно сделать невиртуальной в производном классе (не рекомендуется)
- Тип возвращаемого значения должен совпадать или быть **ковариантным**
- **Ковариантность**: если Base::f() возвращает Base*, то Derived::f() может возвращать Derived*
- Не могут быть шаблонными функциями
- Могут быть дружественными функциями (но теряют виртуальность)
- Могут быть inline (но inline игнорируется для виртуальных вызовов)

Пример ковариантности

```
// Базовый класс
class Animal {
public:
    virtual Animal* clone() const {
        return new Animal(*this);
    }
    virtual ~Animal() {}
};

// Производный класс с ковариантным
// возвращаемым типом
class Dog : public Animal {
public:
    // Ковариантность:
    // возвращает Dog* вместо Animal*
    Dog* clone() const override {
        return new Dog(*this);
    }
};
```

```
// Использование
Dog* dog = new Dog();
Dog* clonedDog = dog->clone();  // Возвращает
Dog*, не нужно dynamic_cast

// Без ковариантности:
class Cat : public Animal {
public:
    // Должен возвращать Animal*
    Animal* clone() const override {
        return new Cat(*this);
    }
};

Cat* cat = new Cat();
Animal* clonedAnimal = cat->clone();  //
Возвращает Animal*
Cat* clonedCat =
dynamic_cast<Cat*>(clonedAnimal);  // Нужен
dynamic_cast
```

```
// Ковариантность работает только с указателями и ссылками
// и только если типы в иерархии наследования
```

Пример «Графическое окно»

```
#include <iostream>
#include <vector>

struct Point {
   int x, y;
   Point(int x = 0, int y = 0) : x(x), y(y) {}
};

class Window {
public:
   virtual void draw() {
      std::cout << "Window::draw() - базовое окно" << std::endl;
   }
   virtual void open() {
      std::cout << "Window::open()" << std::endl;
   }
   virtual void close() {
      std::cout << "Window::close()" << std::endl;
   }
   virtual void move(int x, int y) {
      top_left = Point(x, y);
      std::cout << "Window::move() to (" << x << ", " << y << ")"
<< std::endl;
   }
   virtual ~Window() {}
protected:
   Point top_left;
   Point bottom_right; };
```

```
class Window_with_border : public Window {
public:
   void draw() override {
      Window::draw();
      draw_border();
   }
   void open() override {
      std::cout << "Window_with_border::open()" << std::endl;
   }

private:
   void draw_border() {
      std::cout << "  ┌──────────────────────────┐" << std::endl;
      std::cout << "  │     РАМКА ОКНА           │" << std::endl;
      std::cout << "  └──────────────────────────┘" << std::endl;
   }
};

class Window_with_menu : public Window {
public:
   void draw() override {
      Window::draw();
      draw_menu();
   }

private:
   void draw_menu() {
      std::cout << "  [Файл] [Правка] [Вид] [Справка]" << std::endl;
   }
};
```

```
int main() {
    std::vector<Window*> windows;
    windows.push_back(new Window_with_border());
    windows.push_back(new Window_with_menu());

    std::cout << "=== Рисуем окна ===" << std::endl;
    for (auto w : windows) {
        w->draw();
        std::cout << std::endl;
    }

    std::cout << "=== Открываем окна ===" << std::endl;
    for (auto w : windows) {
        w->open();
    }

    // Освобождаем память
    for (auto w : windows) {
        delete w;
    }

    return 0;
}
```

Time:	Memory:
0.0000 secs	3.568 Mb

Your Output

```
=== Рисуем окна ===
Window::draw() - базовое окно

┌──────────────────┐
│     РАМКА ОКНА   │
└──────────────────┘

Window::draw() - базовое окно
  [Файл] [Правка] [Вид] [Справка]

=== Открываем окна ===
Window_with_border::open()
Window::open()
```

Современные особенности (C++11/14)

Новые возможности в C++11/14:

- override: явное указание переопределения
- final: запрет дальнейшего переопределения или наследования
- Контроль генерации функций: =default, =delete
- Улучшенная производительность vtable в некоторых компиляторах

Итоги:

- Полиморфизм — основа объектно-ориентированного программирования в C++
- Виртуальные функции реализуют динамический полиморфизм через vtable
- Виртуальные деструкторы необходимы для безопасного удаления полиморфных объектов
- Абстрактные классы определяют интерфейсы через чисто виртуальные функции
- Порядок создания объектов критически важен для правильной работы vptr
- Использование override и final (C++11) повышает безопасность кода
- Избегайте вызовов виртуальных функций в конструкторах и деструкторах

Рекомендуемая литература:

1. Бьёрн Страуструп: «Программирование. Принципы и практика использования C++» (2-е изд.)
2. Стэнли Липпман и др.: «Язык программирования C++. Базовый курс» (5-е изд.)
3. Скотт Мейерс: «Эффективный и современный C++»